

Mount Google Drive

```
from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

Load dataset

```
import pandas as pd

df = pd.read_csv('/content/drive/MyDrive/Invistico_Airline.csv')
df.head()
```

	satisfaction	Customer Type	Age	Type of Travel	Class	Flight Distance	Seat comfort	Departure/Arrival time convenient	For
0	satisfied	Loyal Customer	65	Personal Travel	Eco	265	0	0	
1	satisfied	Loyal Customer	47	Personal Travel	Business	2464	0	0	
2	satisfied	Loyal Customer	15	Personal Travel	Eco	2138	0	0	
3	satisfied	Loyal Customer	60	Personal Travel	Eco	623	0	0	
4	satisfied	Loyal Customer	70	Personal Travel	Eco	354	0	0	

5 rows × 22 columns

Understanding the data

Double-click (or enter) to edit

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 129880 entries, 0 to 129879
Data columns (total 22 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   satisfaction                          129880 non-null object
1   Customer Type                        129880 non-null object
2   Age                                  129880 non-null int64
3   Type of Travel                       129880 non-null object
4   Class                                129880 non-null object
5   Flight Distance                      129880 non-null int64
6   Seat comfort                         129880 non-null int64
```

```
7  Departure/Arrival time convenient 129880 non-null int64
8  Food and drink                    129880 non-null int64
9  Gate location                     129880 non-null int64
10 Inflight wifi service              129880 non-null int64
11 Inflight entertainment             129880 non-null int64
12 Online support                     129880 non-null int64
13 Ease of Online booking             129880 non-null int64
14 On-board service                   129880 non-null int64
15 Leg room service                   129880 non-null int64
16 Baggage handling                  129880 non-null int64
17 Checkin service                    129880 non-null int64
18 Cleanliness                       129880 non-null int64
19 Online boarding                    129880 non-null int64
20 Departure Delay in Minutes         129880 non-null int64
21 Arrival Delay in Minutes          129487 non-null float64
dtypes: float64(1), int64(17), object(4)
memory usage: 21.8+ MB
```

checking missing value count

```
df.isnull().sum()
```

	0
satisfaction	0
Customer Type	0
Age	0
Type of Travel	0
Class	0
Flight Distance	0
Seat comfort	0
Departure/Arrival time convenient	0
Food and drink	0
Gate location	0
Inflight wifi service	0
Inflight entertainment	0
Online support	0
Ease of Online booking	0
On-board service	0
Leg room service	0
Baggage handling	0
Checkin service	0
Cleanliness	0
Online boarding	0
Departure Delay in Minutes	0
Arrival Delay in Minutes	393

dtype: int64

Check the median value of the "Arrival Delay in Minutes" variable

```
df['Arrival Delay in Minutes'].describe()
```

Arrival Delay in Minutes	
count	129487.000000
mean	15.091129
std	38.465650
min	0.000000
25%	0.000000
50%	0.000000
75%	13.000000
max	1584.000000

dtype: float64

Fill the missing values in the "Arrival delay in Minutes column" with median

```
df['Arrival Delay in Minutes'] = (
    df['Arrival Delay in Minutes']
    .fillna(df['Arrival Delay in Minutes'].median())
)
```

Identify Features

df.columns

```
Index(['satisfaction', 'Customer Type', 'Age', 'Type of Travel', 'Class',
      'Flight Distance', 'Seat comfort', 'Departure/Arrival time convenient',
      'Food and drink', 'Gate location', 'Inflight wifi service',
      'Inflight entertainment', 'Online support', 'Ease of Online booking',
      'On-board service', 'Leg room service', 'Baggage handling',
      'Checkin service', 'Cleanliness', 'Online boarding',
      'Departure Delay in Minutes', 'Arrival Delay in Minutes'],
      dtype='object')
```

Get the proportion of the category under the "satisfaction" variable

```
df['satisfaction'].value_counts(normalize=True)*100
```

proportion	
satisfaction	
satisfied	54.73283
dissatisfied	45.26717

dtype: float64

Get a quick summary of the counts and percentage of the target variable:(satisfaction)

```
print("Unique values:", df['satisfaction'].unique())
print("\nClass counts:")
print(df['satisfaction'].value_counts())

print("\nClass percentages:")
print(df['satisfaction'].value_counts(normalize=True) * 100)
```

Unique values: ['satisfied' 'dissatisfied']

Class counts:

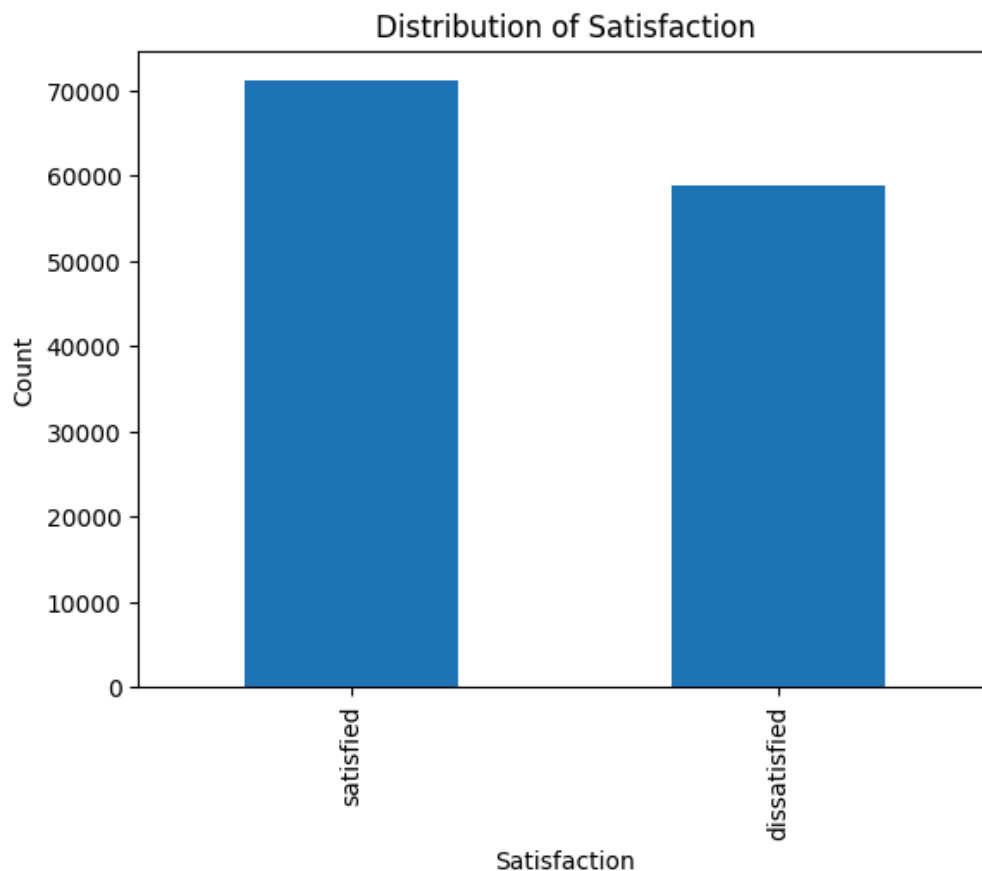
```
satisfaction
satisfied      71087
dissatisfied   58793
Name: count, dtype: int64
```

Class percentages:

```
satisfaction
satisfied      54.73283
dissatisfied    45.26717
Name: proportion, dtype: float64
```

```
import matplotlib.pyplot as plt
```

```
df['satisfaction'].value_counts().plot(kind='bar')
plt.title('Distribution of Satisfaction')
plt.xlabel('Satisfaction')
plt.ylabel('Count')
plt.show()
```



Interpretation

The dataset contains 129,880 observations in total. Satisfied customers constitute 54.73% of the dataset, while dissatisfied customers account for 45.27%. The difference between the two classes is approximately 9.47 percentage points. The classes are reasonably balanced, with neither class overwhelmingly dominating the dataset. Because the class distribution is fairly even, severe class imbalance is not a concern. Therefore, standard classification algorithms such as Logistic Regression, Decision Trees, Random Forests, and KNN can be trained without immediately requiring special imbalance-handling techniques such as oversampling, undersampling, or class weighting.

Conclusion

The target variable (satisfaction) is a binary categorical variable with a relatively balanced distribution. Approximately 55% of passengers are satisfied and 45% are dissatisfied, indicating that the dataset is suitable for building and evaluating classification models without significant concerns about class imbalance.

Identify categorical predictors

```
df.select_dtypes(include='object').columns  
  
Index(['satisfaction', 'Customer Type', 'Type of Travel', 'Class'], dtype='object')
```

Separate the target variable

```
X = df.drop('satisfaction', axis=1)  
y = df['satisfaction']
```

Encode the categorical predictors

```
X_encoded = pd.get_dummies(X, drop_first=True)
```

Encode the target variable(satisfaction)

```
y = y.map({  
    'dissatisfied': 0,  
    'satisfied': 1  
})
```

Verify the result

```
X_encoded.head()
```

	Age	Flight Distance	Seat comfort	Departure/Arrival time convenient	Food and drink	Gate location	Inflight wifi service	Inflight entertainment	Online support
0	65	265	0	0	0	2	2	4	0
1	47	2464	0	0	0	3	0	2	0
2	15	2138	0	0	0	3	2	0	0
3	60	623	0	0	0	3	3	4	0
4	70	354	0	0	0	3	4	3	0

5 rows × 22 columns

X_encoded.info()

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 129880 entries, 0 to 129879
Data columns (total 22 columns):
#   Column                                     Non-Null Count  Dtype
---  -
0   Age                                       129880 non-null  int64
1   Flight Distance                         129880 non-null  int64
2   Seat comfort                           129880 non-null  int64
3   Departure/Arrival time convenient      129880 non-null  int64
4   Food and drink                         129880 non-null  int64
5   Gate location                          129880 non-null  int64
6   Inflight wifi service                  129880 non-null  int64
7   Inflight entertainment                 129880 non-null  int64
8   Online support                         129880 non-null  int64
9   Ease of Online booking                 129880 non-null  int64
10  On-board service                      129880 non-null  int64
11  Leg room service                      129880 non-null  int64
12  Baggage handling                      129880 non-null  int64
13  Checkin service                      129880 non-null  int64
14  Cleanliness                          129880 non-null  int64
15  Online boarding                      129880 non-null  int64
16  Departure Delay in Minutes            129880 non-null  int64
17  Arrival Delay in Minutes              129880 non-null  float64
18  Customer Type_disloyal Customer       129880 non-null  bool
19  Type of Travel_Personal Travel        129880 non-null  bool
20  Class_Eco                            129880 non-null  bool
21  Class_Eco Plus                       129880 non-null  bool
dtypes: bool(4), float64(1), int64(17)
memory usage: 18.3 MB
```

Split data into training and testing sets

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X_encoded,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
```

Scale the predictors using StandardScaler

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

Linear REgression Assumption

Binary Outcome Variable The target variable (satisfaction) contains two categories: Satisfied and Dissatisfied. This satisfies the binary outcome requirement for Logistic Regression.

Missing Data

The dataset was inspected for missing values. The only variable with missing observations was Arrival Delay in Minutes, and these missing values were imputed using the median due to the highly skewed distribution of the variable.

Independent Observations

Each row in the dataset represents an individual passenger survey response and is assumed to be independent of other observations.

Feature Encoding

Categorical variables were converted into numerical format using one-hot encoding, making them suitable for Logistic Regression.

Feature Scaling

Continuous predictors were standardized using StandardScaler to improve model convergence and optimization performance.

Linearity of the Log-Odds

Logistic Regression assumes a linear relationship between continuous predictors and the log-odds of the outcome variable. A formal Box-Tidwell test was not conducted as part of this project; therefore, this assumption was not explicitly verified and should be investigated in future work.

Fit the model

```
from sklearn.linear_model import LogisticRegression

log_reg = LogisticRegression(
    max_iter=1000,
    random_state=42
)

log_reg.fit(X_train_scaled, y_train)
```

LogisticRegression

```
LogisticRegression(max_iter=1000, random_state=42)
```

Generate the predicted probabilities for the "satisfied" class

```
y_pred = log_reg.predict(X_test_scaled)

y_prob = log_reg.predict_proba(X_test_scaled)[: , 1]
#Display the first 10 predicted probabilities
pd.Series(y_prob).head(10)
```

```

      0
0  0.813647
1  0.732018
2  0.041859
3  0.974406
4  0.209819
5  0.375147
6  0.514426
7  0.102175
8  0.108982
9  0.396510

dtype: float64
```

Generate the confusion matrix

```
from sklearn.metrics import confusion_matrix

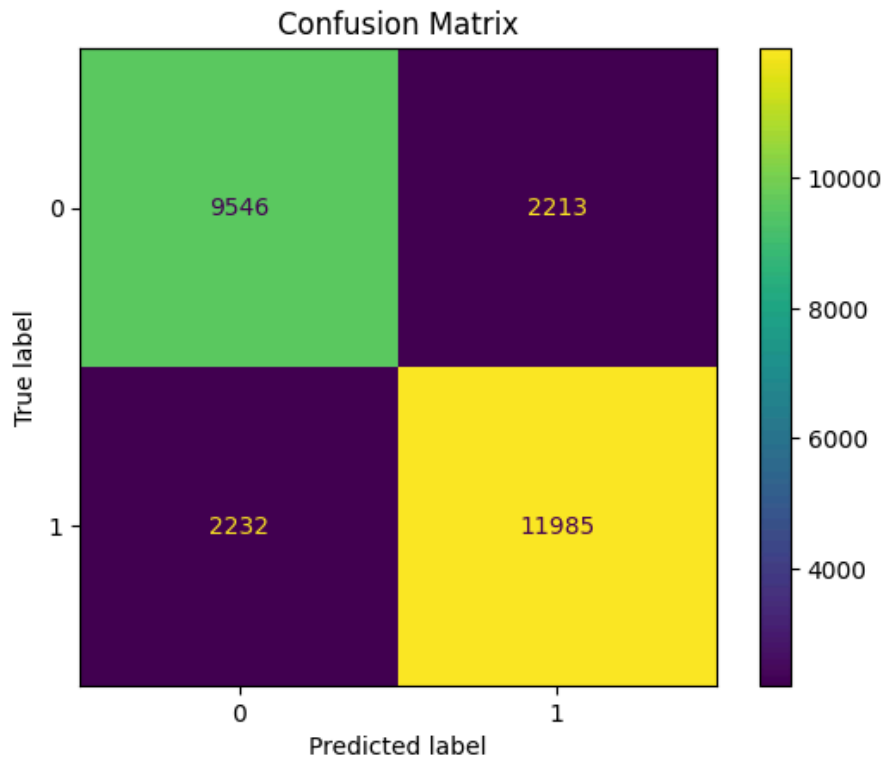
cm = confusion_matrix(y_test, y_pred)
print(cm)
```

```
[[ 9546  2213]
 [ 2232 11985]]
```

Confusion Matrix Plot

```
from sklearn.metrics import ConfusionMatrixDisplay
import matplotlib.pyplot as plt
disp = ConfusionMatrixDisplay(confusion_matrix=cm)
disp.plot()

plt.title("Confusion Matrix")
plt.show()
```



Visualizing the Confusion Matrix The confusion matrix provides a detailed breakdown of correct and incorrect classifications, allowing evaluation of true positives, true negatives, false positives, and false negatives.

Calculate Precision and Recall

```
from sklearn.metrics import precision_score, recall_score

precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)

print("Precision:", precision)
print("Recall:", recall)
```

```
Precision: 0.84413297647556
Recall: 0.8430048533445874
```

Generate a classification Report

```
from sklearn.metrics import classification_report

print(classification_report(y_test, y_pred))
```

	precision	recall	f1-score	support
0	0.81	0.81	0.81	11759
1	0.84	0.84	0.84	14217
accuracy			0.83	25976
macro avg	0.83	0.83	0.83	25976

weighted avg	0.83	0.83	0.83	25976
--------------	------	------	------	-------

Model Evaluation Summary

Metric	Value
Accuracy	83%
Precision	84.4%
Recall	84.3%
F1-Score	84%

Interpretation

The model achieved balanced performance across both classes. Precision and recall were both approximately 84%, indicating that the model effectively identifies satisfied passengers while maintaining a relatively low false positive rate.

Evaluate the F1-Score

```
from sklearn.metrics import f1_score

f1 = f1_score(y_test, y_pred)

print("F1-Score:", f1)
```

```
F1-Score: 0.8435685377441492
```

Interpretation

The model achieved an F1-score of approximately 0.84, indicating a strong balance between precision and recall.

Determine the key co-efficients(factor)

```
coef_df = pd.DataFrame({
    'Feature': X_train.columns,
    'Coefficient': log_reg.coef_[0]
})

coef_df.sort_values(by='Coefficient', ascending=False)
```

	Feature	Coefficient	
7	Inflight entertainment	0.972905	
2	Seat comfort	0.392582	
10	On-board service	0.387346	
13	Checkin service	0.355612	
9	Ease of Online booking	0.334382	
11	Leg room service	0.308623	
15	Online boarding	0.181621	
5	Gate location	0.162930	
8	Online support	0.144692	
12	Baggage handling	0.109271	
16	Departure Delay in Minutes	0.073041	
14	Cleanliness	0.064025	
6	Inflight wifi service	-0.127214	
0	Age	-0.132652	
1	Flight Distance	-0.181013	
21	Class_Eco Plus	-0.197960	
17	Arrival Delay in Minutes	-0.261873	
4	Food and drink	-0.296070	
3	Departure/Arrival time convenient	-0.328774	
19	Type of Travel_Personal Travel	-0.349626	
20	Class_Eco	-0.354649	
18	Customer Type_disloyal Customer	-0.725012	

Model performance Interpretation

The Logistic Regression model achieved an overall accuracy of approximately 83%, indicating strong predictive performance on unseen passenger data.

The model's precision of 84.4% means that when the model predicts a passenger is satisfied, it is correct approximately 84% of the time.

The recall of 84.3% indicates that the model successfully identifies approximately 84% of all genuinely satisfied passengers.

The classification report shows balanced performance across both classes, with F1-scores of 0.81 for dissatisfied passengers and 0.84 for satisfied passengers. This suggests that the model is not heavily biased toward either class and provides reliable predictions of customer satisfaction.

The model achieved a precision of 84.4%, meaning that when a passenger is predicted to be satisfied, the prediction is correct 84.4% of the time. The recall of 84.3% indicates that the model successfully

identifies 84.3% of truly satisfied passengers. Since precision and recall are both high and balanced, the model demonstrates a good trade-off between minimizing false positives and false negatives

Interpret Model coefficient to Identify Key factors of satisfaction

In Logistic Regression:

Positive coefficients increase the likelihood of a passenger being Satisfied. Negative coefficients decrease the likelihood of a passenger being Satisfied. The larger the coefficient (in absolute value), the stronger its influence.

Strongest Positive Drivers of satisfaction

Feature	Coefficient
Inflight entertainment	0.973
Seat comfort	0.393
On-board service	0.387
Checkin service	0.356
Ease of Online booking	0.334
Leg room service	0.309

Interpretation

Inflight entertainment emerged as the strongest positive predictor of customer satisfaction. This suggests that passengers who rate the airline's entertainment services highly are substantially more likely to be satisfied.

Other important positive drivers include:

Seat comfort On-board service Check-in service Ease of online booking Leg room service

These findings indicate that both the in-flight experience and the customer journey before boarding play a major role in determining satisfaction levels.

Strongest Negative Drivers of Satisfaction

Feature	Coefficient
Customer Type_disloyal Customer	-0.725
Class_Eco	-0.355
Type of Travel_Personal Travel	-0.350
Departure/Arrival time convenient	-0.329
Food and drink	-0.296
Arrival Delay in Minutes	-0.262

Interpretation

The strongest negative predictor was being a disloyal customer, indicating that non-loyal customers are significantly less likely to be satisfied.

Other factors associated with lower satisfaction include:

Traveling in Economy Class
Traveling for personal rather than business purposes
Longer arrival delays
Lower ratings for food and beverage services
Lower ratings for departure and arrival time convenience

The negative coefficient for Arrival Delay in Minutes confirms that delays adversely affect customer satisfaction.

Business Recommendation Based on Model Insights

1. Continue Investing in Inflight Entertainment

"Inflight entertainment exhibited the strongest positive relationship with satisfaction and should therefore be prioritized for service improvement initiatives." The airline should continue improving entertainment offerings by:

Expanding movie and television selections
Improving screen quality and reliability
Providing personalized entertainment options

This is likely to generate the greatest improvement in overall customer satisfaction.

2. Enhance Passenger Comfort

Seat comfort and leg room service were among the most influential positive factors.

The airline should:

Upgrade seating quality where feasible
Improve seat ergonomics
Offer additional legroom options

Improving passenger comfort can directly increase satisfaction levels.

3. Improve Service Quality Across Customer Touchpoints

The model highlights the importance of:

- On-board service
- Check-in service
- Online booking

Management should continue investing in staff training and digital service improvements to ensure a seamless customer experience from booking through arrival.

4. Reduce Flight Delays

Arrival delays negatively affect customer satisfaction.

The airline should:

- Improve operational efficiency
- Optimize scheduling
- Enhance delay communication procedures

Even when delays are unavoidable, proactive communication can help mitigate dissatisfaction.

5. Focus on Economy-Class Passengers

Passengers traveling in Economy Class were less likely to be satisfied.

The airline should consider:

- Improving economy cabin comfort
- Enhancing meal quality
- Providing additional value-added services

Since economy passengers constitute a large portion of airline customers, even modest improvements could have a significant impact.

6. Strengthen Customer Loyalty Programs

Disloyal customers were substantially less likely to be satisfied.

The airline should:

- Expand loyalty and rewards programs
- Offer personalized promotions
- Encourage repeat bookings through targeted incentives

Improving customer retention may increase both satisfaction and long-term profitability.

Overall Conclusion

The Logistic Regression model achieved approximately 83% accuracy, with 84.4% precision and 84.3% recall, demonstrating strong predictive performance. The most important drivers of satisfaction were inflight entertainment, seat comfort, on-board service, and check-in service, while disloyal customer status, economy-class travel, personal travel, and arrival delays were associated with lower satisfaction. These insights provide clear opportunities for the airline to improve customer experience and increase overall passenger satisfaction.

Limitations

1. Model assumptions

Logistic Regression assumes a linear relationship between predictors and the log-odds of satisfaction, which may not fully capture complex customer behaviors.

2. Limited feature set

The model only uses variables available in the dataset. Other factors influencing satisfaction, such as ticket price, weather disruptions, or airport conditions, were not included.

3. Historical data

The model was trained on historical observations and may not fully reflect future changes in customer preferences.

4. Correlation vs causation

The model identifies associations with satisfaction but does not prove that a variable directly causes satisfaction

Could not connect to the reCAPTCHA service. Please check your internet connection and reload to get a reCAPTCHA challenge.